

ERC-20 직접 구현

대체 가능 토큰 표준 7개 함수
approve 위임 패턴과 OpenZeppelin 활용

Remix IDE

ERC-20

OZ v5

01

코인 vs 토큰

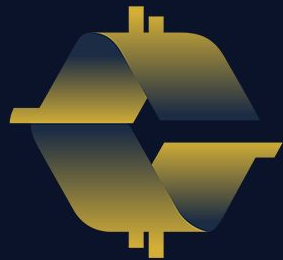
먼저 짚고 가기

"잔액이 어디 있나"가 핵심 구분이다

Coin

Token

4 slides



COINCRAFT
ACADEMY

코인 vs 토큰 — 먼저 짚고 가기 (1/4)

코인(Coin) = 그 블록체인의 "네이티브 화폐"

코인 (Coin)

블록체인 프로토콜 자체가 발행·관리하는
네이티브 화폐. 별도 컨트랙트 없이 존재.

- ETH — 이더리움 네이티브
- BTC — 비트코인 네이티브
- BNB — BNB Chain 네이티브
- MATIC — Polygon 네이티브

특징

- 프로토콜 레벨에서 관리
- 가스비(Gas Fee) 납부 수단
- 스마트 컨트랙트 없이 전송 가능
- 잔액 = 블록체인 상태 트리(State Trie)

→ 노드가 직접 추적

ETH 전송의 내부 동작:

- `msg.sender` 잔액을 프로토콜이 직접 차감 → `to` 잔액에 더함
- 스마트 컨트랙트 호출 없음 — 순수 프로토콜 레이어 연산

gas = 코인으로만 납부: 이더리움에서 모든 트랜잭션은 ETH(코인)로 가스를 낸다.
토큰(ERC-20)으로는 가스를 낼 수 없다 — ETH 지갑이 반드시 필요한 이유

코인 vs 토큰 — 먼저 짚고 가기 (2/4)

토큰(Token) = 그 블록체인 "위에 얹은" 화폐

토큰 (Token)

스마트 컨트랙트 안에 정의된 화폐.
블록체인 위에서 돌아가는 프로그램이다.

- USDT — 이더리움 위 달러 페그 토큰
- LINK — 이더리움 위 오라클 토큰
- UNI — 이더리움 위 거버넌스 토큰
- KBT — KyoboToken (우리가 만드는 것)

특징

- 컨트랙트 코드가 규칙을 정의
- 전송 = 컨트랙트 함수 호출 (트랜잭션)
- 잔액 = 컨트랙트 내부 mapping

```
mapping(address => uint256) _balances
```

→ 컨트랙트가 직접 추적

토큰 전송의 내부 동작:

`transfer(to, amount)` 호출 → 컨트랙트가 `_balances[msg.sender] -= amount` 실행

→ `_balances[to] += amount` 실행 → `Transfer` 이벤트 emit

이 과정에서 ETH(코인)는 이동하지 않는다 — 가스비로만 사용됨

코인 vs 토큰 — 먼저 짚고 가기 (3/4)

동작 방식의 핵심 차이

구분	코인 (ETH)	토큰 (ERC-20)
잔액 위치	블록체인 State Trie (프로토콜)	컨트랙트 <code>_balances</code> mapping
전송 방법	프로토콜 레이어 직접 처리	<code>transfer()</code> 함수 호출 (트랜잭션)
가스비 납부	코인으로 납부	가스비는 여전히 ETH(코인)로
발행 권한	프로토콜 규칙 (채굴/스테이킹)	컨트랙트 코드가 결정
표준	없음 (각 체인 고유)	ERC-20 (공통 인터페이스)

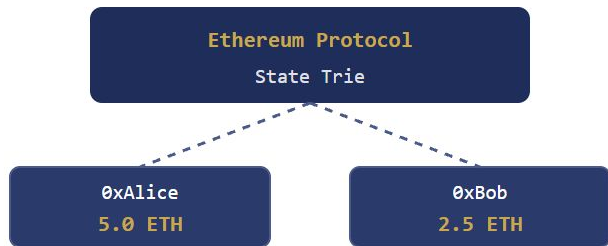
왜 토큰이 필요한가?

ETH는 하나뿐이다. 프로젝트마다 자체 화폐를 발행하려면 컨트랙트로 만들어야 한다.
USDT(달러 페그), UNI(거버넌스), LINK(오라클 결제) — 모두 각자의 용도로 설계된 토큰

코인 vs 토큰 — 먼저 짚고 가기 (4/4)

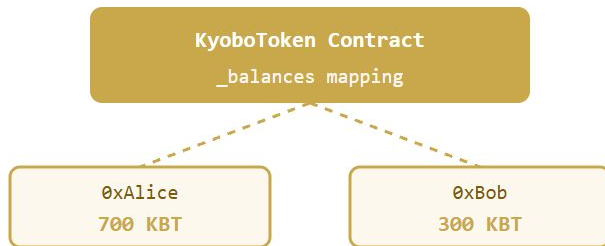
"잔액이 어디 있나"가 핵심 구분이다

코인 (ETH)



프로토콜이 직접 관리

토큰 (ERC-20)



컨트랙트가 직접 관리

핵심 질문: "이 잔액은 누가 보관하는가?" → 프로토콜이면 코인, 컨트랙트면 토큰

02

ERC-20 표준

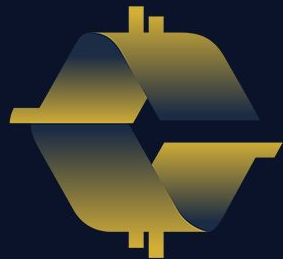
7개 함수 · 2개 이벤트

approve + transferFrom 위임 패턴

Fungible Token

Allowance

6 slides



COINCRAFT
ACADEMY

ERC-20이란 — 대체 가능 토큰 표준

"표준"이란 무슨 뜻인가 · 대체 가능(Fungible)의 의미

표준(Standard)이란?

- 모든 ERC-20 토큰이 같은 함수명·시그니처를 구현
- 거래소는 단 하나의 통합 코드로 모든 토큰 상장
- 지갑·DEX·분석 도구가 코드 수정 없이 연동
- 2015년 Fabian Vogelsteller, GitHub 이슈 #20

대체 가능(Fungible)이란?

- 1 KBT = 1 KBT (동일 가치, 구분 불가)
- 1 KRW = 1 KRW — 화폐의 기본 속성
- NFT(ERC-721): 토큰 #1 ≠ 토큰 #2 (각각 고유)
- 잔액으로 관리 (NFT는 tokenId로 관리)

왜 지금 ERC-20을 배우는가?

KyoboNFT는 ERC-1155. ERC-1155는 ERC-20의 `balanceOf` / `transfer` 개념을 그대로 확장한다.

"표준이 어떻게 설계되는지" 이해하면 ERC-721, ERC-1155가 자연스럽게 연결된다

ERC-20이 바꾼 것: 2017년 ICO 붐 — 이더리움 트래픽의 70% 이상이 ERC-20 전송.

현재 이더리움 상위 100개 토큰 대부분이 ERC-20 기반 (USDT, USDC, LINK, UNI 등)

ERC-20 표준 6개 함수 + 2개 이벤트

EIP-20 명세에 정의된 최소 인터페이스

VIEW — 조회 (3개)

```
// 총 발행량
function totalSupply() external view returns (uint256);

// 특정 주소의 잔액
function balanceOf(address account) external view
    returns (uint256);

// spender가 owner 대신 쓸 수 있는 한도
function allowance(address owner, address spender)
    external view returns (uint256);
```

WRITE — 상태 변경 (3개)

```
// 직접 전송 - msg.sender → to
function transfer(address to, uint256 amount)
    external returns (bool);

// 위임 한도 설정
function approve(address spender, uint256 amount)
    external returns (bool);

// 위임 전송 - from 대신 spender가 실행
function transferFrom(address from, address to,
    uint256 amount) external returns (bool);
```

2개 이벤트 (표준 — 이름·시그니처 변경 불가):

event Transfer(address indexed from, address indexed to, uint256 value) — 잔액 이동 시

event Approval(address indexed owner, address indexed spender, uint256 value) — 위임 한도 설정 시

mint: from=address(0) · burn: to=address(0) — 이더스캔이 이 관례로 "Mint" 탭을 표시

approve + transferFrom — 위임 패턴

왜 이게 필요한가 — transfer만으로 안 되는 이유

문제: DEX(탈중앙 거래소)가 사용자 대신 토큰을 이동해야 한다.

`transfer` 는 `msg.sender` 만 보낼 수 있다 — DEX 컨트랙트가 직접 사용자 토큰을 꺼낼 수 없다

해결: 2단계 위임 패턴

① **approve**

사용자 → DEX
100 KBT 허락

허락 기록

allowance

[user][DEX] = 100

DEX가 호출

② **transferFrom**

DEX → 사용자 대신
실제 이동

① `user.approve(DEX_ADDRESS, 100)`

→ `allowance[user][DEX] = 100`

② `DEX.transferFrom(user, DEX, 100)`

→ `allowance[user][DEX]` 100 차감 (`100 → 0`)

→ `user` 잔액 100 감소, `DEX` 잔액 100 증가

approve = "최대 한도 위임" — 즉시 이동이 아님.

DEX가 `transferFrom` 을 호출해야 비로소 토큰이 이동한다 — 이 구분이 DeFi 전반의 기반

approve + transferFrom — 위임 패턴 (1/3)

allowance — 누가 누구에게 얼마나 허락했나

```
// allowance: 이중 mapping - [owner][spender] = 허용량
mapping(address => mapping(address => uint256)) private _allowances;

// 조회: account2가 account1에게 얼마나 허락했나?
allowance(account2_addr, account1_addr) // → 500
```

allowance 매핑 구조

owner	spender	허용량
Account2	Account1(DEX)	500
Account2	Account3	0

- **owner** = 토큰 보유자 (허락하는 쪽)
- **spender** = 위임받는 쪽 (DEX, 컨트랙트)

transferFrom 내부 로직

```
function transferFrom(from, to, amount) {
    uint256 current = _allowances[from][msg.sender];
    require(current >= amount,
        "Insufficient allowance");
    // 한도 차감
    _allowances[from][msg.sender] = current - amount;
    // 실제 이동
    _transfer(from, to, amount);
}
```

msg.sender = transferFrom을 호출한 spender (DEX 컨트랙트)
한도 차감 후 _transfer 내부 함수로 실제 이동 — transfer와 코드 공유

approve + transferFrom — 위임 패턴 (2/3)

approve 경쟁 조건(race condition) 취약점

공격 시나리오: Alice가 Bob(DEX) 한도를 100→200으로 올리려는 사이 Bob이 먼저 소비

- ① Alice → approve(Bob, 100) → allowance[Alice][Bob] = 100
- ② Bob → transferFrom 50 소비 → allowance[Alice][Bob] = 50
- ③ Alice → approve(Bob, 200) ← 댄스에 들어옴 (아직 미실행)
- ④ Bob → 댄스 모니터링 → 더 높은 gas로 transferFrom(50) 먼저 실행
→ allowance[Alice][Bob] = 0
- ⑤ Alice의 approve(Bob, 200) 실행 → allowance[Alice][Bob] = 200
- ⑥ Bob → transferFrom(200) 추가 소비
→ Bob 총 획득: 50 + 50 + 200 = 300 (Alice 의도는 200)

왜 가능한가: approve 는 현재값에 더하는 게 아니라 새 값으로 덮어쓴다.

덮어쓰기 직전 남은 한도를 공격자가 먼저 소진 → 새 한도도 추가 사용

방어 패턴

```
// 패턴 ①: 0으로 리셋 후 새 한도
approve(Bob, 0);    // 먼저 초기화
approve(Bob, 200);  // 새 한도 설정
// 최악: 0 + 200 = 200 (의도와 일치)
```

```
// 패턴 ②: increaseAllowance (OZ)
// 기존 50 남았을 때 150 추가
increaseAllowance(Bob, 150);
// → 50 + 150 = 200 (현재값 기준 증가)
// ※ OZ v5에서는 deprecated
```

approve + transferFrom — 위임 패턴 (3/3)

Phase 1 연관성 — KyoboNFT와의 연결

ERC-20 approve 패턴

```
approve(spender, amount)
```

- owner가 spender에게 amount 한도 위임
- `allowance[owner][spender]` 에 기록
- spender가 `transferFrom` 으로 실행

ERC-1155 setApprovalForAll 패턴

```
setApprovalForAll(operator, true)
```

- owner가 operator를 전체 전송 위임
- `_operatorApprovals[owner][operator]` 기록
- operator가 `safeTransferFrom` 으로 실행

KyoboNFT Phase 1 시나리오:

보험 가입자(owner)가 VASP 서버를 operator로 승인

→ `setApprovalForAll(VASP_SERVER, true)`

→ VASP 서버가 `safeTransferFrom` 으로 NFT 전송 처리

패턴은 ERC-20 approve+transferFrom과 동일 — operator 범위만 다름

구분	ERC-20	ERC-1155
위임 함수	<code>approve(spender, amount)</code>	<code>setApprovalForAll(operator, bool)</code>
위임 범위	특정 금액	모든 tokenId 전체
실행 함수	<code>transferFrom</code>	<code>safeTransferFrom</code>

03

ERC-20 직접 구현

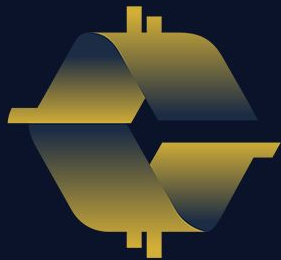
KyoboToken — AccessControl + Pausable

7개 함수를 처음부터 작성한다

KyoboToken

mint · burn

6 slides



COINCRAFT
ACADEMY

ERC-20 직접 구현 (1/6) — 상태변수

역할 · name · symbol · decimals · totalSupply · 두 개의 mapping

```
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/utils/Pausable.sol";

contract KyoboToken is AccessControl, Pausable {
    // — 역할 —
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");

    // — 상태변수 —
    string public name;
    string public symbol;
    uint8 public decimals; // 소수점 자릿수. ETH 기준 18
    uint256 public totalSupply;

    mapping(address => uint256) private _balances;
    mapping(address => mapping(address => uint256)) private _allowances;
```

decimals = 18: ETH와 동일 정밀도. 1 KBT = 10^{18} 최소 단위 (wei 단위)

· USDC = 6 (달러 소수점 6자리) · WBTC = 8 (BTC 소수점 맞춤)

· UI 표시: `amount / 1018` → 사람이 읽는 숫자. 컨트랙트 내부는 항상 정수 연산

ERC-20 직접 구현 (2/6) — 이벤트 + 생성자

Transfer · Approval 이벤트 (ERC-20 표준 — 이름 변경 불가)

```
// — 이벤트 —————  
// ERC-20 표준 이벤트: 이름·시그니처 변경 불가  
event Transfer(address indexed from, address indexed to, uint256 value);  
event Approval(address indexed owner, address indexed spender, uint256 value);  
  
// — 생성자 —————  
constructor(string memory _name, string memory _symbol) {  
    name      = _name;  
    symbol     = _symbol;  
    decimals  = 18; // 1 KBT = 1_000_000_000_000_000_000  
  
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);  
    _grantRole(MINTER_ROLE,      msg.sender);  
    _grantRole(PAUSER_ROLE,      msg.sender);  
}
```

Transfer 이벤트 관례 (ERC-20 명세에 없지만 업계 표준):

- emit Transfer(address(0), to, amount) → **mint** (from=0 = "무에서 생성")
- emit Transfer(from, address(0), amount) → **burn** (to=0 = "소각")

이더스캔·The Graph·Dune이 이 관례로 "Mint" 탭을 별도 표시한다

ERC-20 직접 구현 (3/6) — 조회 + 발행 + 소각

balanceOf · allowance · mint · burn

```
// — 조회 —  
function balanceOf(address account) public view returns (uint256) {  
    return _balances[account];  
}  
  
function allowance(address owner, address spender) public view returns (uint256) {  
    return _allowances[owner][spender];  
}  
  
// — 발행 —  
function mint(address to, uint256 amount) public onlyRole(MINTER_ROLE) whenNotPaused {  
    require(to != address(0), "Mint to zero address");  
    _balances[to] += amount;  
    totalSupply += amount;  
    emit Transfer(address(0), to, amount); // from=0: mint 관례  
}  
  
// — 소각 —  
function burn(uint256 amount) public whenNotPaused {  
    require(_balances[msg.sender] >= amount, "Insufficient balance");  
    _balances[msg.sender] -= amount;  
    totalSupply -= amount;  
    emit Transfer(msg.sender, address(0), amount); // to=0: burn 관례  
}
```

ERC-20 직접 구현 (4/6) — transfer + approve

직접 전송 · 위임 한도 설정

```
// — 전송 —  
  
function transfer(address to, uint256 amount) public whenNotPaused returns (bool) {  
    _transfer(msg.sender, to, amount);  
    return true;  
}  
  
function approve(address spender, uint256 amount) public returns (bool) {  
    require(spender != address(0), "Approve to zero address");  
    _allowances[msg.sender][spender] = amount;  
    emit Approval(msg.sender, spender, amount);  
    return true;  
}
```

transfer:

- whenNotPaused 적용 — 정지 중 전송 불가
- 실제 로직은 _transfer 내부 함수에 위임
- transferFrom 과 코드 공유 (중복 제거)

approve 주의:

- whenNotPaused 없음 — 정지 중에도 승인 가능
- 덮어쓰기 방식 → 경쟁 조건 취약점
- 변경 시 approve(0) 먼저 호출할 것

ERC-20 직접 구현 (5/6) — transferFrom + _transfer

위임 전송 · 내부 공통 함수

```
function transferFrom(address from, address to, uint256 amount)
    public whenNotPaused returns (bool)
{
    uint256 currentAllowance = _allowances[from][msg.sender];
    require(currentAllowance >= amount, "Insufficient allowance");
    _allowances[from][msg.sender] = currentAllowance - amount; // 한도 차감
    _transfer(from, to, amount);
    return true;
}

// — 내부 함수 - transfer / transferFrom 공통 로직 분리 —
function _transfer(address from, address to, uint256 amount) internal {
    require(from != address(0), "Transfer from zero address");
    require(to != address(0), "Transfer to zero address");
    require(_balances[from] >= amount, "Insufficient balance");
    _balances[from] -= amount;
    _balances[to] += amount;
    emit Transfer(from, to, amount);
}
```

_transfer 분리 설계: transfer 와 transferFrom 모두 동일한 이동 로직 공유.
중복 제거 + 수정 시 한 곳만 고치면 됨. OpenZeppelin도 동일 패턴을 따른다.

ERC-20 직접 구현 (6/6) — 긴급 정지 + 설계 결정

pause · unpause · Solidity 0.8.x 오버플로우 보호

```
// — 긴급 정지 —  
function pause() public onlyRole(PAUSER_ROLE) { _pause(); }  
function unpause() public onlyRole(PAUSER_ROLE) { _unpause(); }  
}
```

whenNotPaused 적용 범위

- `mint` — 정지 중 발행 불가
- `burn` — 정지 중 소각 불가
- `transfer` — 정지 중 전송 불가
- `transferFrom` — 정지 중 위임 전송 불가
- `approve` — 정지 중에도 승인 가능

Solidity 0.8.x — SafeMath 불필요

0.8.0부터 정수 연산에 오버플로우·언더플로우 검사 기본 내장.

- 2018년 BEC 토큰 사고 계기
- 2^{255} 오버플로우로 수천억 피해
- `unchecked { }` 블록에서만 생략 가능
- 별도 SafeMath import 불필요

실습 확인 포인트 요약: `mint` → `transfer` → `approve` → `transferFrom` → `burn` → `pause` → `transfer` 실패

04

OZ 활용 & 확장 표준

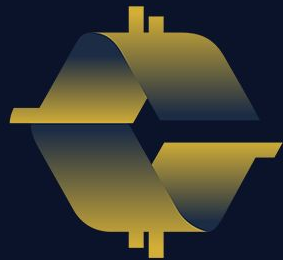
직접 구현 vs OpenZeppelin ERC20

ERC-2612 Permit · ERC-4626 Vault

OZ ERC20

ERC-2612

4 slides



COINCRAFT
ACADEMY

OpenZeppelin ERC20 — 직접 구현 vs OZ 활용 (1/2)

OZ ERC20 상속 버전 코드

```
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/utils/Pausable.sol";

contract KyoboTokenOZ is ERC20, AccessControl, Pausable {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");

    constructor() ERC20("KyoboToken", "KBT") {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(MINTER_ROLE, msg.sender);
        _grantRole(PAUSER_ROLE, msg.sender);
    }

    function mint(address to, uint256 amount) public onlyRole(MINTER_ROLE) {
        _mint(to, amount); // OZ 내부 함수 호출
    }

    function pause() public onlyRole(PAUSER_ROLE) { _pause(); }
    function unpause() public onlyRole(PAUSER_ROLE) { _unpause(); }

    // Pausable + ERC20 연결 - OZ v5: _beforeTokenTransfer → _update로 변경
    function _update(address from, address to, uint256 value) internal override {
        if (paused()) revert EnforcedPause();
        super._update(from, to, value);
    }
}
```

OpenZeppelin ERC20 — 직접 구현 vs OZ 활용 (2/2)

직접 구현한 코드와 비교

항목	직접 구현 (KyoboToken)	OZ 활용 (KyoboTokenOZ)
상태변수	<code>name·symbol·decimals·totalSupply·_balances·_allowances</code> 직접 선언	OZ 내부 처리 (불필요)
transfer/approve/transferFrom	직접 구현 (~30줄)	OZ 내부 구현 (상속)
mint / burn	직접 구현	<code>_mint()</code> / <code>_burn()</code> 호출
Pause 연결	<code>whenNotPaused</code> 각 함수에 직접	<code>_update</code> override 1곳
전체 코드량	~80줄	~30줄
보안 감사	직접 검증 필요	OZ 감사 완료

OZ를 쓰는 이유: 표준 로직은 검증된 코드에 맡기고 비즈니스 로직만 추가한다.

역할 제어(MINTER_ROLE) + 일시정지 연결 — 이것만 작성하면 ERC-20 완성

⚠ OZ v4 → v5 주의: `_beforeTokenTransfer` → `_update` 로 변경됨.

v4 코드를 검색해서 그대로 붙이면 컴파일 오류. 버전 확인 필수

ERC-2612 (Permit) — 가스리스 승인

오프체인 서명으로 approve 트랜잭션을 없앤다

기존 approve 흐름 (트랜잭션 2번)

- ① Alice → `approve(DEX, 100)` **트랜잭션 ①**
가스비 발생
- ② DEX → `transferFrom(Alice, DEX, 100)` **트랜잭션 ②**
가스비 발생

= 트랜잭션 2번, 가스 2배

ERC-2612 Permit 흐름 (트랜잭션 1번)

- ① Alice → EIP-712 서명 생성 **가스 없음**
오프체인 서명만 (트랜잭션 없음)
- ② DEX → 서명 포함해서 `permit()` 호출
→ `approve` + `transferFrom` 1번에 처리 **트랜잭션 1번**

= 트랜잭션 1번, 가스 절반

채택 현황: USDC v2 · DAI · Uniswap v2/v3 LP 토큰

DeFi UX 개선의 핵심 — 사용자가 approve 트랜잭션 없이 바로 스왑 가능

OZ: `import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol"`

ERC-4626 (Vault) — 수익형 금고 표준

DeFi yield vault의 공통 인터페이스 — ERC-20의 확장



프로토콜	Vault 토큰	설명
Aave v3	aToken (aUSDC 등)	예치 시 1:1로 aToken 발행, 이자가 잔액에 반영
Compound	cToken (cUSDC 등)	예치 시 cToken 발행, 환율로 수익 반영
Yearn Finance	yvToken	전략적 yield farming 수익 자동 복리

왜 표준이 필요했나: 각 프로토콜마다 인터페이스가 달랐다 → 통합 코드 작성 어려움
ERC-4626으로 `deposit` / `withdraw` / `redeem` / `previewDeposit` 표준화 (2022년)

다음 세션 예고 — ERC-721: tokenId가 고유하고 수량이 1인 NFT.
ERC-20의 `balanceOf(address)` → ERC-721의 `ownerOf(tokenId)` 로 관점이 바뀐다

실습 — KyoboToken 배포 + 6단계 시나리오

Remix IDE · 생성자 인자: name="KyoboToken", symbol="KBT"

```
[1] mint(Account2, 1000)
    → Transfer(0x0 → Account2, 1000) 이벤트
    → totalSupply → 1000
    → balanceOf(Account2) → 1000

[2] Account2 → transfer(Account3, 300)
    → Transfer(Account2 → Account3, 300) 이벤트
    → balanceOf(Account2) → 700
    → balanceOf(Account3) → 300

[3] Account2 → approve(Account1주소, 500)
    → Approval 이벤트 확인
    → allowance(Account2주소, Account1주소) → 500
```

```
[4] Account1 → transferFrom(Account2주소, Account3주소, 200)
    → allowance(Account2, Account1) → 300 (500-200)
    → balanceOf(Account2) → 500
    → balanceOf(Account3) → 500

[5] Account2 → burn(100)
    → Transfer(Account2 → 0x0, 100) 이벤트
    → totalSupply → 900

[6] Account1 → pause()
    → Account2 → transfer(...) → EnforcedPause() revert
    → Account1 → unpause() → 전송 재개 확인
```

이벤트 확인 포인트: Remix Logs에서 각 단계마다 `from` / `to` / `value` 값 직접 확인

[1] `from = 0x000...000` (mint 관례) · [5] `to = 0x000...000` (burn 관례)